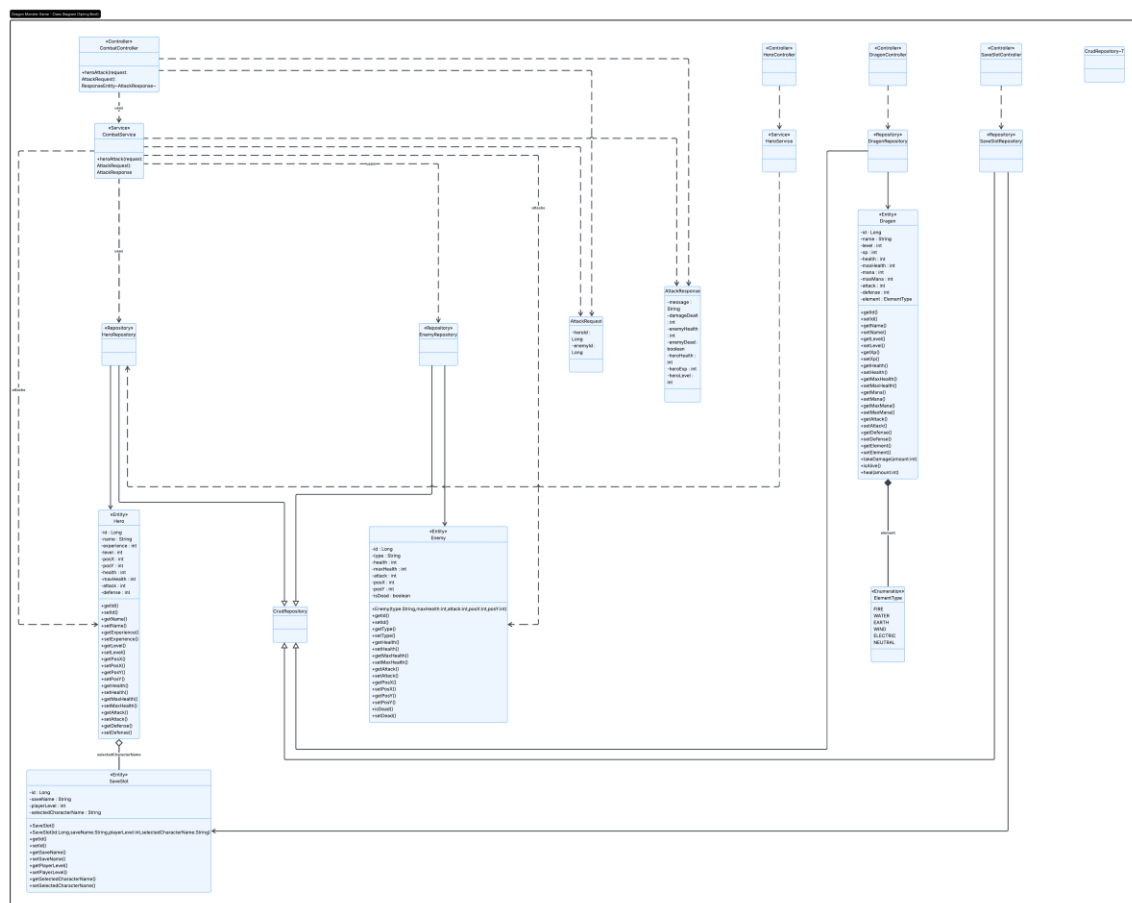


1. Skema Class Diagram

Class Diagram Dragon Monster Game menggambarkan struktur kelas pada aplikasi beserta hubungan antar kelas yang membangun sistem permainan. Arsitektur yang digunakan menerapkan Layered Architecture pada framework Spring Boot, sehingga setiap komponen memiliki tanggung jawab yang berbeda sesuai dengan prinsip Separation of Concerns (SoC). Dengan pendekatan ini, kode menjadi lebih modular, mudah dipelihara, serta mempermudah proses pengembangan maupun pengujian aplikasi.

Secara umum, diagram kelas dibagi menjadi lima lapisan utama, yaitu Controller Layer, Service Layer, Repository Layer, DTO Layer, dan Entity/Model Layer.



2. Penjelasan Lengkap Class Diagram

A. Entity / Model Layer

Lapisan Entity merupakan representasi objek bisnis yang dipetakan langsung ke tabel pada database menggunakan Java Persistence API (JPA) melalui anotasi seperti `@Entity`, `@Table`, dan `@Id`. Setiap kelas pada lapisan ini menyimpan data utama yang digunakan oleh sistem permainan.

1. Hero

Kelas Hero merupakan entitas utama yang menyimpan seluruh informasi karakter yang dikendalikan oleh pemain.

Atribut yang dimiliki meliputi:

- id
- name
- experience
- level
- posisi (posX dan posY)
- health
- maxHealth
- attack
- defense

Selain atribut tersebut, kelas Hero menyediakan method getter dan setter untuk melakukan manipulasi data karakter selama permainan berlangsung.

Hero menjadi pusat data pemain karena hampir seluruh proses permainan, seperti pertarungan, penyimpanan permainan (save game), dan peningkatan level, menggunakan data dari kelas ini.

2. Enemy

Kelas Enemy merepresentasikan musuh yang muncul di dalam permainan.

Data yang disimpan antara lain:

- id
- type
- health
- maxHealth
- attack

- posisi
- status isDead

Selain menyediakan getter dan setter, kelas Enemy memiliki constructor yang digunakan ketika sistem membuat musuh baru berdasarkan jenis musuh tertentu.

Enemy digunakan oleh sistem pertarungan (CombatService) untuk menghitung damage serta menentukan apakah musuh telah dikalahkan.

3. Dragon

Kelas Dragon merupakan entitas yang menyimpan data naga atau boss monster.

Data yang disimpan meliputi:

- nama naga
- level
- pengalaman (XP)
- health
- mana
- attack
- defense
- element

Selain getter dan setter, kelas Dragon memiliki beberapa fungsi bisnis, yaitu:

- takeDamage()
- heal()
- isAlive()

Method tersebut memungkinkan objek Dragon melakukan perubahan kondisi secara langsung ketika terjadi pertarungan.

4. SaveSlot

Kelas SaveSlot digunakan sebagai media penyimpanan perkembangan permainan (save game).

Informasi yang disimpan meliputi:

- nama save
- level pemain

- karakter yang dipilih

Dengan adanya SaveSlot, pemain dapat melanjutkan permainan dari kondisi terakhir yang telah disimpan.

5. ElementType

ElementType merupakan kelas Enumeration (Enum) yang mendefinisikan seluruh elemen yang dimiliki naga.

Nilai yang tersedia yaitu:

- FIRE
- WATER
- EARTH
- WIND
- ELECTRIC
- NEUTRAL

Penggunaan Enumeration membuat data elemen menjadi lebih konsisten karena hanya nilai yang telah didefinisikan yang dapat digunakan.

B. Repository Layer

Repository Layer bertugas sebagai penghubung antara aplikasi dengan database.

Seluruh repository merupakan interface yang mewarisi (extends) CrudRepository, sehingga operasi CRUD seperti:

- Create
- Read
- Update
- Delete

dapat dilakukan tanpa perlu menuliskan query SQL secara manual.

Repository yang digunakan terdiri dari:

- HeroRepository
- Mengelola seluruh data Hero.
- EnemyRepository
- Mengelola data musuh selama permainan berlangsung.

- DragonRepository
- Mengelola data master Dragon.
- SaveSlotRepository
- Mengelola data penyimpanan permainan.

Dengan menggunakan Spring Data JPA, seluruh repository memperoleh implementasi CRUD secara otomatis.

C. Service Layer

Service Layer merupakan lapisan yang menjalankan business logic pada aplikasi.

Berbeda dengan Repository yang hanya berkomunikasi dengan database, Service berisi aturan permainan.

CombatService

CombatService merupakan inti dari sistem pertarungan.

Class ini menerima data AttackRequest dari Controller, kemudian:

- mengambil data Hero dari HeroRepository,
- mengambil data Enemy dari EnemyRepository,
- menghitung damage berdasarkan atribut Hero dan Enemy,
- memperbarui HP musuh,
- menambahkan pengalaman apabila musuh berhasil dikalahkan,
- mengembalikan hasil pertarungan dalam bentuk AttackResponse.

Dengan demikian seluruh logika permainan tersimpan pada satu tempat sehingga mudah dipelihara.

HeroService

HeroService bertanggung jawab terhadap seluruh operasi yang berhubungan dengan Hero, seperti:

- mengambil data Hero,
- menyimpan Hero,
- memperbarui Hero,
- menghapus Hero.

HeroService menjadi penghubung antara HeroController dan HeroRepository.

D. DTO Layer (Data Transfer Object)

DTO digunakan sebagai media pertukaran data antara client dan server.

Penggunaan DTO bertujuan agar data yang dikirim melalui REST API tidak langsung menggunakan Entity.

1. AttackRequest

AttackRequest berisi informasi yang dikirim pemain ketika menyerang musuh.

Atribut yang dimiliki yaitu:

- heroId
- enemyId

2. AttackResponse

AttackResponse merupakan hasil yang dikirim kembali kepada client setelah proses pertarungan selesai.

Data yang dikirim meliputi:

- pesan hasil pertarungan,
- damage yang diberikan,
- HP musuh,
- status musuh,
- HP Hero,
- pengalaman Hero,
- level Hero.

Dengan penggunaan DTO, komunikasi API menjadi lebih aman dan efisien.

E. Controller Layer

Controller merupakan lapisan paling atas yang menerima HTTP Request dari client.

Controller tidak berisi logika permainan, melainkan hanya menerima request, memanggil Service, kemudian mengembalikan hasil kepada client dalam bentuk ResponseEntity.

Controller yang digunakan yaitu:

1. CombatController

Menangani seluruh proses pertarungan.

Request yang diterima berupa AttackRequest.

Controller akan memanggil CombatService dan mengembalikan AttackResponse.

2. HeroController

Mengelola seluruh operasi Hero melalui REST API.

3. DragonController

Mengelola data Dragon.

4. SaveSlotController

Mengelola proses penyimpanan maupun pemuatan data permainan.

F. Pola Relasi Antar Kelas

Class Diagram Dragon Monster Game menerapkan beberapa jenis hubungan antar kelas.

1. Dependency

Dependency menunjukkan bahwa suatu kelas menggunakan layanan dari kelas lain.

Contohnya:

- CombatController menggunakan CombatService.
- HeroController menggunakan HeroService.
- CombatService menggunakan HeroRepository dan EnemyRepository.

Hubungan ini menunjukkan bahwa Controller bergantung pada Service, sedangkan Service bergantung pada Repository.

2. Inheritance

Seluruh Repository mewarisi interface CrudRepository.

Contohnya:

- HeroRepository → CrudRepository
- EnemyRepository → CrudRepository
- DragonRepository → CrudRepository
- SaveSlotRepository → CrudRepository

Dengan inheritance tersebut, operasi CRUD tersedia secara otomatis.

3. Composition

Dragon memiliki hubungan composition dengan ElementType.

Artinya setiap objek Dragon selalu memiliki satu ElementType sebagai atribut yang menentukan elemen naga.

4. Aggregation

Hero memiliki hubungan aggregation dengan SaveSlot.

Satu Hero dapat memiliki beberapa SaveSlot yang digunakan untuk menyimpan perkembangan permainan.

5. Association

CombatService memiliki hubungan association dengan Hero dan Enemy karena proses pertarungan melibatkan kedua entitas tersebut.